

Phase 1: Real-Time Chat App

Flutter Tech Stack, Architecture and Development Task List

Primary goal	Build a polished WhatsApp-style one-to-one messaging application.
Platforms	iOS and Android from one Flutter codebase.
Primary login	Email and password.
Optional later login	Phone number.
Discovery	Search users by a unique username.
Phase 1 scope	Authentication, profiles, username search and real-time text chat.

Phase 1 milestone: Two users can register, select unique usernames, find each other, start a conversation and exchange messages in real time.

1. Phase 1 Product Scope

Phase 1 should deliberately avoid voice commands, group calls and complex assistant features. The goal is to establish a reliable, secure and visually polished messaging foundation first.

Included now	Email authentication; unique usernames; user profiles; username search; one-to-one chat; typing indicators; online status; read receipts; push notifications; smooth animations; dark mode.
Later phases	Phone authentication; media messages; voice notes; group chat; audio and video calls; voice navigation; emergency commands; AI assistant actions.

2. Recommended Tech Stack

Layer	Technology	Purpose
Mobile framework	Flutter	One codebase for iOS and Android with strong UI and animation support.
Language	Dart	Type-safe language used by Flutter.
State management	Riverpod	Predictable, testable state management with dependency injection.
Navigation	GoRouter	Declarative routing, route guards and deep-link support.
Backend	Supabase	Authentication, PostgreSQL, real-time subscriptions and file storage.
Database	PostgreSQL	Reliable relational storage for users, conversations, messages and receipts.
Authentication	Supabase Auth	Email login, verification, password reset and optional phone login later.
Real-time layer	Supabase Realtime	Live message delivery, typing state and presence updates.
File storage	Supabase Storage	Profile photographs now and media messages in a later phase.
Local cache	Drift	Offline-friendly local database and paginated message cache.
Secure storage	flutter_secure_storage	Securely stores tokens and session information.
Data models	Freezed + json_serializable	Immutable models, generated parsing and safer state objects.
Push notifications	Firebase Cloud Messaging	Background and terminated-app message notifications.
Crash reporting	Firebase Crashlytics	Production crash reporting and error visibility.
Animations	Flutter animations + Rive/Lottie	Lightweight motion, typing animations and polished transitions.
Testing	flutter_test + integration_test + mocktail	Unit, widget and end-to-end test coverage.
Repository	GitHub	Open-source collaboration, issues, pull requests and documentation.

Layer	Technology	Purpose
CI	GitHub Actions	Automated formatting, static analysis, testing and build checks.

3. High-Level Architecture

Flutter application: screens, Riverpod state, local Drift cache and real-time subscriptions.

Supabase: authentication, PostgreSQL database, Row Level Security, Realtime and Storage.

Firebase: push notifications and crash reporting.

GitHub: source control, issues, contribution workflow and continuous integration.

Recommended Flutter folder structure

```
lib/  
  app/ - app setup, router and themes  
  core/ - common services, constants, errors and widgets  
  features/  
    authentication/  
    onboarding/  
    profile/  
    user_search/  
    conversations/  
    messaging/  
    settings/  
  main.dart
```

4. Core Database Tables

Table	Important fields
profiles	id, username, username_normalized, display_name, avatar_url, bio, phone_number, is_online, last_seen_at, created_at
conversations	id, type, created_at, updated_at, last_message_id
conversation_members	conversation_id, user_id, joined_at, last_read_message_id, is_muted
messages	id, conversation_id, sender_id, content, message_type, reply_to_message_id, created_at, edited_at, deleted_at
message_receipts	message_id, user_id, delivered_at, read_at
user_devices	id, user_id, fcm_token, platform, updated_at

Important: Enforce a unique index on *username_normalized*. Search usernames using normalized lowercase values while preserving the user's selected display casing.

5. Development Task List

A. Project Foundation

Priority	Task
P0	Create the Flutter project with iOS and Android targets.
P0	Configure development, staging and production environments.
P0	Add Riverpod, GoRouter, Freezed, Drift and secure-storage packages.
P0	Create app themes, typography, spacing rules and reusable components.
P0	Configure lint rules and code generation.
P1	Add dark mode and system-theme support.

B. Supabase and Security

Priority	Task
P0	Create the Supabase project and version-controlled database migrations.
P0	Create profiles, conversations, members, messages and receipt tables.
P0	Enable Row Level Security on every table containing user data.
P0	Write policies so users can access only permitted profiles and conversations.
P0	Create indexes for username search, conversation ordering and message pagination.
P0	Create .env.example and ignore all real environment files.
P0	Keep service-role keys only on trusted backend infrastructure.

C. Authentication

Priority	Task
P0	Build email registration and login screens.
P0	Add email verification.
P0	Add forgot-password and reset-password flows.
P0	Persist sessions securely and restore login when the app launches.
P1	Add logout from the current device and all devices.
P2	Prepare optional phone-login architecture without blocking Phase 1.

D. Username and Profile

Priority	Task
P0	Require a unique username after registration.
P0	Normalize usernames and validate allowed characters.
P0	Build profile creation and editing screens.

Priority	Task
P0	Add display name, biography and profile image.
P1	Show online status and last seen with privacy controls.

E. User Search and Chat Creation

Priority	Task
P0	Build username search with debounce.
P0	Support exact and prefix username matching.
P0	Hide private email addresses and phone numbers from search results.
P0	Open a user's profile from a search result.
P0	Create or reuse an existing one-to-one conversation.
P1	Add recent searches.

F. Real-Time Messaging

Priority	Task
P0	Build a conversation list ordered by latest activity.
P0	Build the chat screen with paginated message history.
P0	Send and receive text messages in real time.
P0	Use optimistic UI so outgoing messages appear immediately.
P0	Add sent, delivered and read states.
P0	Add unread counts and last-message previews.
P0	Add typing indicators.
P1	Add reply, edit and delete message actions.
P1	Add retry handling for failed sends.
P1	Cache recent messages locally with Drift.

G. Motion and Visual Polish

Priority	Task
P1	Add subtle message-bubble entrance animation.
P1	Add animated typing dots.
P1	Add smooth profile-to-chat Hero transitions.
P1	Add swipe-to-reply.
P1	Add animated unread badges and skeleton loading placeholders.
P1	Add light haptic feedback for sending and important actions.

Priority	Task
P1	Keep most transitions between 150 and 300 milliseconds.
P1	Respect reduced-motion accessibility settings.

H. Notifications

Priority	Task
P1	Integrate Firebase Cloud Messaging.
P1	Store and refresh device notification tokens.
P1	Show notifications while the application is in the background.
P1	Open the correct conversation when a notification is selected.
P1	Avoid duplicate notifications while the same chat is already open.

I. Quality and Open Source

Priority	Task
P0	Add unit tests for authentication, username validation and repositories.
P0	Add widget tests for login, search and chat screens.
P1	Add an integration test for registration, search, messaging and read receipts.
P0	Configure GitHub Actions for formatting, analysis and tests.
P0	Write a README with setup instructions and screenshots.
P0	Add CONTRIBUTING.md, CODE_OF_CONDUCT.md and issue templates.
P0	Add SECURITY.md with private vulnerability-reporting instructions.
P1	Create repository labels such as good first issue, bug, enhancement and documentation.

6. Suggested Delivery Milestones

Milestone	Deliverable
M1 - Foundation	Flutter project, themes, routing, environments, Supabase connection and CI.
M2 - Accounts	Email registration, login, verification, password reset and secure sessions.
M3 - Identity	Unique usernames, profiles, profile images and username search.
M4 - Core Chat	One-to-one conversation creation and real-time text messaging.
M5 - Messaging Quality	Read receipts, typing indicators, pagination, local cache and error recovery.
M6 - Polish	Animations, haptics, dark mode, accessibility and loading states.
M7 - Release Ready	Push notifications, testing, documentation and open-source contribution setup.

7. Definition of Done for Phase 1

Phase 1 is complete when two users can register with email, verify their accounts, choose unique usernames, find each other through username search, start one conversation and exchange real-time text messages. Messages must survive application restarts, show delivery and read states, support notifications and remain protected by database security policies.

Recommended first-release exclusions

Do not block the first public release on phone authentication, group chat, media sharing, calls, voice commands or AI assistant actions. These features should be built after the messaging foundation is stable.